Technical Architecture

Private Login — Zero-Knowledge Identity Infrastructure

Version 1.0 — December 2025

System Overview

Private Login consists of three primary components that work together to enable zero-knowledge identity verification on Midnight: Smart Contracts (Compact), the Developer SDK (TypeScript), and the Credential Wallet (React). The system leverages Midnight's dual-state architecture to maintain private credentials while enabling public verification.

High-Level Architecture

The architecture follows a layered approach:

```
┌─────────────────────────────────────────────────────────────┐
│                    PRESENTATION LAYER                       │
│    [ Credential Wallet ]    [ Integrated dApps ]            │
└─────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
┌─────────────────────────────────────────────────────────────┐
│                     SDK LAYER (TypeScript)                  │
│    [ PrivateLogin SDK ]  [ React Components ]               │
│    [ Midnight.js ]       [ Wallet Connector ]               │
└─────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
┌─────────────────────────────────────────────────────────────┐
│                    CONTRACT LAYER (Compact)                 │
│    [ Credential Registry ]  [ Verification Engine ]         │
│    [ Policy Evaluator ]     [ Proof Validator ]             │
└─────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
┌─────────────────────────────────────────────────────────────┐
│                      MIDNIGHT NETWORK                       │
│    [ Private State ]       [ Public State ]                 │
│    [ Proof Server ]        [ Impact VM ]                    │
└─────────────────────────────────────────────────────────────┘
```

Core Components

1. Smart Contracts (Compact)

Four primary contracts handle identity operations:

CredentialRegistry

Manages credential issuance, storage references, and revocation status.

- **Private State:** Credential data (encrypted), user mappings
- **Public State:** Credential hashes, revocation flags, issuer signatures
- **Key Functions:** issueCredential(), revokeCredential(), getCredentialStatus()

VerificationEngine

Processes verification requests and generates/validates proofs.

- **Private State:** Credential values being verified
- **Public State:** Verification results (boolean), request metadata
- **Key Functions:** requestVerification(), submitProof(), getVerificationResult()

PolicyEvaluator

Evaluates complex verification policies combining multiple conditions.

- **Private State:** Policy evaluation intermediate values
- **Public State:** Policy definitions, evaluation results
- **Key Functions:** registerPolicy(), evaluatePolicy(), updatePolicy()

IssuerRegistry

Manages trusted credential issuers and their permissions.

- **Public State:** Issuer addresses, credential types they can issue, trust scores
- **Key Functions:** registerIssuer(), authorizeCredentialType(), getIssuerInfo()

2. Developer SDK (TypeScript)

The SDK provides a clean API for dApp developers to integrate identity verification without handling ZK complexity.

Module Structure

Module	Purpose
`@privatelogin/core`	Core verification logic, policy builders, type definitions
`@privatelogin/react`	React components: VerifyButton, CredentialGate, hooks
`@privatelogin/wallet`	Wallet connection, credential management, proof generation
`@privatelogin/contracts`	Contract ABIs, deployment addresses, type-safe bindings

Key Classes

- **PrivateLoginClient:** Main entry point, handles initialization and configuration
- **Verifier:** Creates and manages verification requests
- **PolicyBuilder:** Fluent API for building complex verification policies
- **CredentialManager:** Manages credential storage and retrieval in wallet
- **ProofGenerator:** Interfaces with Midnight proof server

3. Credential Wallet

The consumer-facing application where users manage their verifiable credentials. Built as a Chrome extension (Phase 1) with potential web app expansion.

Core Features

- **Credential Storage:** Encrypted local storage with optional cloud backup
- **Verification Requests:** Review, approve, or deny incoming verification requests
- **Selective Disclosure:** Choose which credential attributes to prove
- **Audit Log:** History of all verifications performed
- **Issuer Connection:** Interface for obtaining new credentials from trusted issuers

UI Components

- **Dashboard:** Overview of credentials and recent activity
- **Credential Detail:** View credential attributes, issuer info, expiration
- **Request Handler:** Modal for approving/denying verification requests
- **Settings:** Wallet connection, backup/restore, privacy preferences

Data Flow: Verification Request

The complete flow for a dApp requesting user verification:

1. **dApp Initiates Request:** Calls SDK with policy (e.g., 'age >= 18')
2. **SDK Creates Verification Request:** Generates unique request ID, encodes policy
3. **Wallet Receives Request:** Popup shows dApp name, requested proof, credential options
4. **User Approves:** Selects credential, confirms disclosure
5. **Proof Generation:** Wallet interfaces with Midnight proof server via SDK
6. **On-Chain Submission:** Proof submitted to VerificationEngine contract
7. **Contract Verification:** Midnight validates ZK proof cryptographically
8. **Result Returned:** Boolean result (verified/not) returned to dApp via SDK
9. **Audit Logged:** Wallet records verification in local history

Credential Schema

Credentials follow a structured schema designed for flexibility and W3C Verifiable Credentials compatibility:

Field	Type	Description
`id`	string	Unique credential identifier (UUID)
`type`	CredentialType	AGE, RESIDENCY, TOKEN_BALANCE, NFT_OWNERSHIP, etc.
`issuer`	Address	Midnight address of credential issuer
`subject`	Address	User's Midnight address
`claims`	Record<string, any>	Credential attributes (e.g., { birthDate: '1990-01-15' })
`issuedAt`	timestamp	When credential was issued
`expiresAt`	timestamp | null	Optional expiration timestamp
`signature`	bytes	Issuer's cryptographic signature

Supported Verification Types

Phase 1 (MVP)

Type	Description & Use Cases
AGE	Prove age >= threshold without revealing birthdate. Use: gambling, alcohol, adult content, age-restricted DeFi.

Phase 2

Type	Description & Use Cases
TOKEN_BALANCE	Prove holding >= X tokens without revealing exact balance. Use: token-gated access, governance eligibility.
NFT_OWNERSHIP	Prove ownership of NFT from collection without revealing which one. Use: exclusive access, community membership.

Phase 3+ (Future)

Type	Description & Use Cases
RESIDENCY	Prove residence in country/region without revealing address. Use: geo-restricted services, tax compliance.
ACCREDITED	Prove accredited investor status without revealing financials. Use: securities offerings, private investments.
CREDENTIAL	Prove professional license/certification without revealing ID. Use: professional services, hiring.

Security Considerations

Threat Model

- **Malicious dApps:** Attempting to extract more information than approved
- **Credential Forgery:** Attempting to create fake credentials
- **Replay Attacks:** Reusing old proofs for new verifications
- **Wallet Compromise:** Attacker gains access to user's credential wallet
- **Issuer Compromise:** Trusted issuer is compromised or malicious

Mitigations

1. **Selective Disclosure:** Users explicitly approve each attribute disclosed
2. **Issuer Signatures:** All credentials cryptographically signed by issuer
3. **Nonce-Based Proofs:** Each proof includes unique nonce, preventing replay
4. **Local Encryption:** Credentials encrypted at rest in wallet
5. **Revocation Checking:** Verification always checks issuer revocation status
6. **Audit Logging:** All verification activity logged for user review

Audit Requirements

Before mainnet launch, the following must be professionally audited:

- All Compact smart contracts
- Proof generation logic in SDK
- Wallet credential storage and encryption
- Inter-component communication protocols

— End of Document —